



UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERÍA

Escuela Profesional de Ingeniería de Sistemas

Informe de Especificación de Requerimientos

Sistema Analizador de Rendimiento de Consultas (Query Analyzer)

Curso: Base de Datos II

Docente: Patrick Cuadros Quiroga

Integrantes:

Carbajal Vargas, Andre Alejandro (2023077287)

Yupa Gómez, Fátima Sofía (2023076618)

Tacna - Perú

2026

CONTROL DE VERSIONES					
Versión	Hecha por	Revisada por	Aprobada por	Fecha	Motivo
1.0	ACV, FYG	ACV, FYG	P. Cuadros Q.	2026-04-29	Versión inicial
1.1	ACV, FYG	ACV, FYG	P. Cuadros Q.	2026-06-23	Actualización factual y formato institucional
1.2	ACV, FYG	ACV, FYG	P. Cuadros Q.	2026-07-04	Ampliación de requisitos, procesos, casos de uso y modelos del sistema

ÍNDICE GENERAL

INTRODUCCIÓN

I. Generalidades de la Empresa

1. Nombre de la Empresa

2. Visión

3. Misión

4. Organigrama

II. Visionamiento de la Empresa

1. Descripción del Problema

2. Objetivos de Negocios

3. Objetivos de Diseño

4. Alcance del proyecto

5. Viabilidad del Sistema

6. Información obtenida del Levantamiento de Información

III. Análisis de Procesos

a) Diagrama del Proceso Actual - Diagrama de actividades

b) Diagrama del Proceso Propuesto - Diagrama de actividades inicial

IV. Especificación de Requerimientos de Software

a) Cuadro de Requerimientos funcionales inicial

b) Cuadro de Requerimientos no funcionales

c) Cuadro de Requerimientos funcionales final

d) Reglas de Negocio

V. Fase de Desarrollo

1. Perfiles de Usuario

2. Modelo Conceptual

a) Diagrama de Paquetes

b) Diagrama de Casos de Uso

c) Escenarios de Caso de Uso (narrativa)

3. Modelo Lógico

a) Análisis de Objetos

b) Diagrama de Actividades con objetos

c) Diagrama de Secuencia

d) Diagrama de Clases

CONCLUSIONES

RECOMENDACIONES

BIBLIOGRAFÍA

WEBGRAFÍA

INTRODUCCIÓN

El presente documento especifica los requerimientos de software del sistema **Analizador de Rendimiento de Consultas (Query Analyzer)**. El proyecto se desarrolla en el contexto académico del curso **Base de Datos II** y tiene como propósito facilitar el análisis de planes de ejecución, métricas observables y evidencia técnica de consultas en múltiples motores de bases de datos.

Query Analyzer responde a una necesidad concreta: los motores SQL, NoSQL, de grafos, series de tiempo y servicios cloud exponen mecanismos de diagnóstico diferentes. Cada motor tiene comandos, formatos y métricas propias, por lo que el usuario debe cambiar de herramienta y criterio constantemente para estudiar el rendimiento de una consulta.

La solución propone una arquitectura local, extensible y segura basada en adaptadores. El sistema permite administrar perfiles, diagnosticar conexiones, ejecutar `EXPLAIN` o su equivalente, construir reportes factuales, conservar el plan original, exportar resultados y, opcionalmente, solicitar una interpretación mediante inteligencia artificial. La IA no reemplaza la evidencia del motor ni modifica las métricas observadas.

Este informe organiza la especificación desde la visión institucional del proyecto hasta los requisitos funcionales, no funcionales, reglas de negocio, casos de uso y modelos lógicos que describen la solución implementada.

I. Generalidades de la Empresa

1. Nombre de la Empresa

Para fines del proyecto académico, la empresa u organización se define como el **Equipo Query Analyzer - Universidad Privada de Tacna**, conformado por estudiantes de la Escuela Profesional de Ingeniería de Sistemas. No se trata de una empresa comercial formal, sino de una unidad académica responsable de analizar, diseñar, implementar y documentar una herramienta de apoyo al diagnóstico de consultas.

2. Visión

Ser una herramienta local, confiable y extensible para inspeccionar planes de ejecución, métricas y evidencia observable de consultas en múltiples motores de bases de datos desde interfaces uniformes como CLI, TUI, API REST, MCP y Visual Studio Code.

3. Misión

Proporcionar a estudiantes, desarrolladores y administradores de bases de datos una forma segura y reproducible de obtener evidencia real del rendimiento de consultas, evitando puntuaciones arbitrarias y separando la interpretación opcional mediante IA de los datos factuales entregados por cada motor.

4. Organigrama

```
flowchart TD
    DOC["Docente del curso<br/>Patrick Cuadros Quiroga"]
    EQ["Equipo Query Analyzer"]
    ARQ["Análisis y arquitectura"]
    DEV["Desarrollo de adaptadores e interfaces"]
    QA["Pruebas, calidad y documentación"]
    USR["Usuarios objetivo<br/>estudiantes, desarrolladores y DBA"]

    DOC --> EQ
    EQ --> ARQ
    EQ --> DEV
    EQ --> QA
    ARQ --> USR
    DEV --> USR
    QA --> USR
```

II. Visionamiento de la Empresa

1. Descripción del Problema

Los usuarios que analizan rendimiento de consultas enfrentan una alta fragmentación técnica. PostgreSQL, MySQL, SQLite, SQL Server, CockroachDB, YugabyteDB, MongoDB, Redis, DynamoDB, Cassandra, Elasticsearch, Neo4j e InfluxDB ofrecen mecanismos distintos para obtener planes, métricas o trazas. Esta situación genera los siguientes problemas:

- aprendizaje repetido de comandos y formatos por motor;
- dificultad para conservar evidencia comparable dentro de un mismo reporte;
- riesgo de interpretar como equivalentes métricas que no tienen la misma semántica;
- exposición accidental de credenciales al usar scripts o herramientas aisladas;
- dependencia de procesos manuales para documentar antes y después de una optimización;
- confusión entre datos reales del motor y recomendaciones generadas por IA o heurísticas.

2. Objetivos de Negocios

ID	Objetivo de negocio	Resultado esperado
ON-01	Reducir el esfuerzo de diagnóstico de consultas	Usar un flujo común para múltiples motores
ON-02	Incrementar la confiabilidad de la evidencia técnica	Conservar plan original, métricas disponibles y fecha de análisis
ON-03	Facilitar el aprendizaje de rendimiento de bases de datos	Mostrar planes y métricas de forma comprensible
ON-04	Mejorar la seguridad de credenciales	Cifrar perfiles y sanitizar mensajes sensibles
ON-05	Favorecer la integración con herramientas modernas	Exponer CLI, TUI, API REST, MCP y extensión VS Code

3. Objetivos de Diseño

ID	Objetivo de diseño	Decisión asociada
OD-01	Mantener bajo acoplamiento entre motores e interfaces	Uso de <code>AdapterRegistry</code> y <code>BaseAdapter</code>
OD-02	Representar planes de forma común sin perder detalle	Uso de <code>PlanNode</code> , <code>raw_plan</code> y <code>metrics</code>
OD-03	Evitar diagnósticos no verificables	No incluir score universal ni antipatronos deterministas centrales

ID	Objetivo de diseño	Decisión asociada
OD-04	Proteger secretos	Uso de perfiles cifrados, <code>SecretStr</code> y sanitización
OD-05	Permitir operación local	API enlazada a <code>127.0.0.1</code> por defecto y MCP por stdio
OD-06	Tolerar ausencia o fallo de IA	<code>AIAnalysisResult</code> opcional y separado del reporte factual

4. Alcance del proyecto

Incluido

- Administración de perfiles locales de conexión.
- Cifrado de credenciales persistidas.
- Diagnóstico de configuración, DNS, TCP, autenticación y operación.
- Registro de motores mediante `AdapterRegistry`.
- Ejecución de `EXPLAIN` o mecanismo equivalente.
- Construcción de `QueryAnalysisReport` factual.
- Normalización parcial del plan mediante `PlanNode`.
- Conservación del plan original del motor.
- Métricas específicas por adaptador.
- Historial local y exportación de reportes.
- Interfaces CLI, TUI y API REST local.
- Servidor MCP con herramienta de análisis.
- Extensión para Visual Studio Code.
- Interpretación opcional mediante proveedores compatibles con API tipo OpenAI.
- Pruebas unitarias, de contrato, integración y documentación.

Fuera de alcance

- Modificar consultas, índices o esquemas automáticamente.
- Reemplazar una plataforma APM o monitoreo continuo.
- Garantizar que una sugerencia de IA mejore el rendimiento.
- Generar una puntuación universal de calidad.
- Comparar como equivalentes métricas incompatibles entre motores.
- Persistir credenciales recibidas por API REST.
- Ejecutar operaciones destructivas como parte del flujo normal.

5. Viabilidad del Sistema

El sistema es viable técnica, operativa y económicamente. La solución utiliza Python, Pydantic, Typer, Rich, Textual, FastAPI, Docker, TypeScript y GitHub Actions, tecnologías disponibles y adecuadas para el alcance académico. Los costos se concentran en el tiempo del equipo, conectividad y energía; no requiere licencias comerciales para el desarrollo ni para su ejecución principal.

El diseño por adaptadores permite extender motores sin reescribir las interfaces. La separación entre datos factuales e IA reduce riesgos de interpretación y mejora la transparencia del

producto.

6. Información obtenida del Levantamiento de Información

Fuente	Información obtenida	Uso en el sistema
Revisión de motores de bases de datos	Cada motor expone planes y métricas con sintaxis distinta	Definición de adaptadores por motor
Revisión de necesidades académicas	Se requiere evidencia reproducible para estudiar rendimiento	Reportes factuales exportables
Pruebas con servicios locales	Algunos motores requieren Docker o emuladores	Separación de pruebas unitarias e integración
Uso de CLI/TUI/API	Los usuarios necesitan distintos niveles de interacción	Interfaces múltiples sobre el mismo núcleo
Revisión de seguridad	Las credenciales deben protegerse y no aparecer en errores	Cifrado, <code>SecretStr</code> y sanitización
Uso de agentes y editores	MCP y VS Code reducen fricción de uso	Servidor MCP y extensión empaquetada

III. Análisis de Procesos

a) Diagrama del Proceso Actual - Diagrama de actividades

El proceso actual representa el trabajo manual antes de contar con Query Analyzer.

```
flowchart TD
    A["Inicio"] --> B["Identificar motor de base de datos"]
    B --> C["Buscar comando de diagnóstico específico"]
    C --> D["Preparar conexión o consola del motor"]
    D --> E["Ejecutar EXPLAIN, PROFILE o herramienta equivalente"]
    E --> F["Copiar salida manualmente"]
    F --> G["Interpretar formato propio del motor"]
    G --> H{"¿Se requiere otro motor?"}
    H -- "Sí" --> B
    H -- "No" --> I["Redactar conclusiones manuales"]
    I --> J["Fin"]
```

Problemas del proceso actual

- No existe un formato común de reporte.
- La evidencia se copia manualmente y puede perder contexto.
- Los secretos pueden quedar expuestos en scripts o capturas.
- Las métricas se interpretan sin reglas claras por motor.
- La comparación entre motores puede inducir errores.

b) Diagrama del Proceso Propuesto - Diagrama de actividades inicial

El proceso propuesto centraliza el análisis mediante Query Analyzer.

```
flowchart TD
    A["Inicio"] --> B["Seleccionar interfaz: CLI, TUI, API, MCP o VS Code"]
    B --> C["Crear o elegir perfil de conexión"]
    C --> D["Diagnosticar conexión"]
    D --> E{"¿Conexión válida?"}
    E -- "No" --> F["Mostrar mensaje sanitizado y corregir perfil"]
    F --> C
    E -- "Sí" --> G["Ingresar consulta"]
    G --> H["Crear adaptador mediante AdapterRegistry"]
    H --> I["Ejecutar EXPLAIN o equivalente"]
    I --> J["Construir QueryAnalysisReport factual"]
    J --> K{"¿IA configurada?"}
    K -- "Sí" --> L["Generar AIAnalysisResult separado"]
    K -- "No" --> M["Continuar sin IA"]
    L --> N["Presentar y exportar reporte"]
    M --> N
    N --> O["Guardar historial local si aplica"]
    O --> P["Fin"]
```

IV. Especificación de Requerimientos de Software

a) Cuadro de Requerimientos funcionales inicial

ID	Requerimiento funcional inicial	Prioridad
RFI-01	Registrar motores de bases de datos mediante un catálogo común	Alta
RFI-02	Crear perfiles de conexión por motor	Alta
RFI-03	Ejecutar análisis de consulta desde línea de comandos	Alta
RFI-04	Obtener plan de ejecución o mecanismo equivalente del motor	Alta
RFI-05	Mostrar resumen del plan y métricas disponibles	Alta
RFI-06	Exportar resultados para documentación	Media
RFI-07	Proteger credenciales almacenadas	Alta
RFI-08	Permitir pruebas con motores locales mediante Docker	Media

b) Cuadro de Requerimientos no funcionales

ID	Requerimiento no funcional	Criterio de aceptación
RNF-01	Ejecutarse con Python 3.14+	El proyecto corre mediante <code>uv run</code> y binarios distribuidos
RNF-02	Mantener formato y lint automatizado	Ruff debe aprobar revisión y formato
RNF-03	Mantener tipado verificable	mypy debe validar el paquete principal
RNF-04	No exponer secretos	Errores y respuestas públicas deben estar sanitizados
RNF-05	Liberar conexiones correctamente	Los adaptadores deben soportar context manager
RNF-06	Separar datos factuales de IA	<code>AIAnalysisResult</code> no modifica métricas del reporte
RNF-07	Operar localmente por defecto	API en <code>127.0.0.1</code> y MCP por stdio

ID	Requerimiento no funcional	Criterio de aceptación
RNF-08	Soportar extensibilidad por motor	Nuevos motores deben integrarse mediante adaptadores
RNF-09	Conservar compatibilidad de reportes	<code>QueryAnalysisReport</code> debe preservar plan original y normalizado
RNF-10	Diferenciar ausencia de datos de valor cero	Valores no disponibles deben quedar como <code>None</code> u omitidos
RNF-11	No calcular score universal	El reporte no debe contener puntuación global de calidad
RNF-12	Proporcionar documentación navegable	GitHub Pages debe renderizar informes y evidencias

c) Cuadro de Requerimientos funcionales final

ID	Requerimiento funcional final	Actor principal	Estado
RFF-01	Registrar y listar motores con <code>AdapterRegistry</code>	Sistema	Implementado
RFF-02	Validar <code>ConnectionConfig</code> por motor	Sistema	Implementado
RFF-03	Crear, listar, consultar, seleccionar y eliminar perfiles	Usuario CLI/TUI	Implementado
RFF-04	Cifrar credenciales de perfiles locales	Sistema	Implementado
RFF-05	Diagnosticar configuración, DNS, TCP, autenticación y operación	Usuario CLI/TUI/API	Implementado
RFF-06	Ejecutar <code>EXPLAIN</code> o mecanismo equivalente por adaptador	Usuario	Implementado
RFF-07	Construir <code>PlanNode</code> cuando exista plan jerárquico	Sistema	Implementado
RFF-08	Construir <code>QueryAnalysisReport</code> factual	Sistema	Implementado
RFF-09	Conservar <code>raw_plan</code> y métricas específicas	Sistema	Implementado
RFF-10	Mostrar resultados en CLI con Rich	Usuario CLI	Implementado

ID	Requerimiento funcional final	Actor principal	Estado
RFF-11	Mostrar resultados en TUI con Textual	Usuario TUI	Implementado
RFF-12	Mantener historial local por perfil	Usuario TUI	Implementado
RFF-13	Exportar reportes en JSON y Markdown	Usuario	Implementado
RFF-14	Exponer API REST bajo <code>/api/v1/analyzer</code>	Cliente HTTP	Implementado
RFF-15	Exponer herramienta MCP <code>analyze_query(query, profile)</code>	Agente compatible	Implementado
RFF-16	Analizar consultas desde Visual Studio Code	Usuario VS Code	Implementado
RFF-17	Ejecutar interpretación IA opcional	Usuario con proveedor IA	Implementado
RFF-18	Soportar 13 motores registrados	Sistema	Implementado
RFF-19	Publicar documentación y evidencias en GitHub Pages	Equipo	Implementado
RFF-20	Generar artefactos de release por plataforma	Equipo	Implementado

Motores soportados

Categoría	Motores
SQL y NewSQL	PostgreSQL, MySQL, SQLite, Microsoft SQL Server, CockroachDB y YugabyteDB
NoSQL	MongoDB, Redis, DynamoDB, Cassandra y Elasticsearch
Grafos	Neo4j
Series de tiempo	InfluxDB

d) Reglas de Negocio

ID	Regla de negocio
RN-01	Todo motor soportado debe registrarse en <code>AdapterRegistry</code> .
RN-02	Todo adaptador debe implementar conexión, desconexión, prueba de conexión y análisis.
RN-03	El motor se normaliza a minúsculas antes de validarse.
RN-04	Solo se aceptan motores incluidos en el catálogo soportado.
RN-05	Todo puerto configurado debe estar entre 1 y 65535.
RN-06	SQLite y DynamoDB no requieren puerto TCP para su configuración mínima.
RN-07	Redis, Cassandra, DynamoDB y Elasticsearch pueden operar sin nombre de base de datos tradicional.
RN-08	SQLite permite contraseña vacía por ser un motor basado en archivo local.
RN-09	CockroachDB puede aceptar contraseña vacía en escenarios locales de desarrollo.
RN-10	Si una contraseña se proporciona para motores que la requieren, no puede quedar vacía.
RN-11	Las credenciales persistidas en perfiles deben cifrarse.
RN-12	La API REST no debe persistir credenciales recibidas en solicitudes.
RN-13	Los mensajes públicos no deben exponer contraseñas, tokens, API keys ni cabeceras Bearer.
RN-14	<code>QueryAnalysisReport.execution_time_ms</code> debe ser mayor que cero.
RN-15	El reporte debe incluir motor, consulta, tiempo de ejecución, resumen, fecha y métricas disponibles.
RN-16	El plan original debe conservarse cuando el motor lo entregue.
RN-17	<code>PlanNode</code> debe usarse solo cuando exista una estructura jerárquica representable.

ID	Regla de negocio
RN-18	La ausencia de una métrica no debe reemplazarse por cero.
RN-19	El sistema no debe calcular una puntuación universal de calidad.
RN-20	Las recomendaciones de IA deben almacenarse en <code>AIAnalysisResult</code> y no mezclarse con métricas factuales.
RN-21	Si la IA no está configurada o falla, el análisis factual debe seguir siendo válido.
RN-22	Las interfaces CLI, TUI, API, MCP y VS Code deben reutilizar el mismo núcleo de análisis.
RN-23	Las operaciones normales de análisis no deben modificar esquemas, índices ni datos del motor.
RN-24	Las pruebas unitarias no deben depender de Docker.
RN-25	Las pruebas de integración que requieren motores reales deben mantenerse separadas.

V. Fase de Desarrollo

1. Perfiles de Usuario

Perfil	Descripción	Necesidades principales
Estudiante de bases de datos	Usuario académico que estudia planes de ejecución	Ejecutar análisis simples, observar planes y exportar evidencia
Desarrollador	Usuario que optimiza consultas durante desarrollo	Diagnosticar perfiles, comparar resultados y usar CLI/VS Code
Administrador de bases de datos	Usuario técnico con conocimiento de motores	Revisar métricas específicas y planes originales
Docente / evaluador	Revisa el cumplimiento del proyecto	Consultar documentación, evidencias y criterios de calidad
Cliente HTTP	Sistema externo que invoca la API	Enviar conexión y consulta para recibir reporte estructurado
Agente compatible con MCP	Asistente que usa herramientas estructuradas	Invocar <code>analyze_query</code> con un perfil local

2. Modelo Conceptual

El modelo conceptual se organiza alrededor de perfiles, adaptadores, reportes y presentación. El usuario no interactúa directamente con drivers específicos; selecciona un perfil o conexión y el sistema delega la ejecución al adaptador correspondiente.

```
flowchart LR
    U["Usuario"] --> P["Perfil de conexión"]
    U --> Q["Consulta"]
    P --> C["ConnectionConfig"]
    C --> A["Adaptador"]
    Q --> A
    A --> E["Motor de base de datos"]
    E --> R["Plan y métricas"]
    R --> REP["QueryAnalysisReport"]
    REP --> PN["PlanNode"]
    REP --> RAW["raw_plan"]
    REP --> MET["metrics"]
    REP --> AI["AIAnalysisResult opcional"]
```

a) Diagrama de Paquetes

```
flowchart TD
    CLI["query_analyzer.cli"]
    TUI["query_analyzer.tui"]
```



```
API["query_analyzer.api"]
MCP["query_analyzer.mcp_server"]
CFG["query_analyzer.config"]
CORE["query_analyzer.core"]
ADP["query_analyzer.adapters"]
SQL["adapters.sql"]
NOSQL["adapters.nosql"]
GRAPH["adapters.graph"]
TS["adapters.timeseries"]
VSC["integrations.vscode-query-analyzer"]
```

```
CLI --> CFG
CLI --> ADP
TUI --> CFG
TUI --> ADP
API --> ADP
API --> CORE
MCP --> CFG
MCP --> ADP
VSC --> API
CORE --> ADP
ADP --> SQL
ADP --> NOSQL
ADP --> GRAPH
ADP --> TS
```

b) Diagrama de Casos de Uso

```
flowchart LR
    EST["Estudiante / Desarrollador / DBA"]
    DOC["Docente"]
    HTTP["Cliente HTTP"]
    AG["Agente MCP"]
    VSC["Usuario VS Code"]
    SYS["Query Analyzer"]

    EST --> UC1["Administrar perfiles"]
    EST --> UC2["Diagnosticar conexión"]
    EST --> UC3["Analizar consulta"]
    EST --> UC4["Exportar reporte"]
    EST --> UC5["Consultar historial"]
    EST --> UC6["Solicitar IA opcional"]
    DOC --> UC7["Revisar documentación y evidencias"]
    HTTP --> UC8["Invocar API REST"]
    AG --> UC9["Invocar herramienta MCP"]
    VSC --> UC10["Analizar selección en VS Code"]

    UC1 --> SYS
    UC2 --> SYS
    UC3 --> SYS
    UC4 --> SYS
    UC5 --> SYS
    UC6 --> SYS
    UC7 --> SYS
    UC8 --> SYS
    UC9 --> SYS
    UC10 --> SYS
```

c) Escenarios de Caso de Uso (narrativa)

CU-01. Administrar perfil de conexión

Campo	Descripción
Actor principal	Usuario CLI/TUI
Propósito	Crear, listar, consultar, seleccionar o eliminar perfiles de conexión
Precondición	El usuario conoce los datos mínimos del motor
Flujo principal	El usuario ingresa motor y credenciales; el sistema valida <code>ConnectionConfig</code> ; cifra secretos; guarda el perfil local
Alternativas	Si el motor no es soportado o el puerto es inválido, se rechaza la configuración
Resultado	Perfil disponible para análisis y diagnóstico

CU-02. Diagnosticar conexión

Campo	Descripción
Actor principal	Usuario CLI/TUI/API
Propósito	Verificar si un perfil o conexión puede operar antes de analizar
Precondición	Existe una configuración de conexión
Flujo principal	El sistema valida configuración, verifica conectividad, prueba autenticación y ejecuta comprobación operativa
Alternativas	Si falla una etapa, se devuelve mensaje claro y sanitizado
Resultado	Diagnóstico con estado, duración y detalle seguro

CU-03. Analizar consulta

Campo	Descripción
Actor principal	Usuario
Propósito	Obtener plan, métricas y resumen factual de una consulta
Precondición	Existe conexión válida y consulta no vacía
Flujo principal	El sistema crea adaptador; conecta; ejecuta <code>EXPLAIN</code> o equivalente; construye <code>QueryAnalysisReport</code> ; desconecta

Campo	Descripción
Alternativas	Si el motor no entrega plan jerárquico, el reporte conserva métricas o datos disponibles
Resultado	Reporte factual listo para visualizar o exportar

CU-04. Exportar reporte

Campo	Descripción
Actor principal	Usuario CLI/TUI
Propósito	Guardar evidencia de análisis en formato reutilizable
Precondición	Existe un <code>QueryAnalysisReport</code> válido
Flujo principal	El usuario elige formato; el sistema serializa a JSON o Markdown
Alternativas	Si el destino no es escribible, se informa error sin perder el reporte en memoria
Resultado	Archivo de reporte disponible para documentación o comparación

CU-05. Solicitar interpretación IA opcional

Campo	Descripción
Actor principal	Usuario con proveedor IA configurado
Propósito	Obtener explicación natural de la evidencia
Precondición	Existen variables o parámetros de proveedor IA
Flujo principal	El sistema envía plan, consulta y motor al proveedor; recibe resumen, observaciones y recomendaciones
Alternativas	Si IA falla, el sistema mantiene el reporte factual sin bloquear el análisis
Resultado	<code>AIAnalysisResult</code> adjunto y separado del reporte factual

CU-06. Usar API REST

Campo	Descripción
Actor principal	Cliente HTTP

Campo	Descripción
Propósito	Integrar análisis de consultas desde otra aplicación
Precondición	API local activa
Flujo principal	El cliente envía conexión y consulta a <code>/api/v1/analyzer/explain</code> ; el sistema responde con reporte estructurado
Alternativas	Si la solicitud es inválida, se devuelve error controlado
Resultado	Respuesta JSON con evidencia factual

CU-07. Usar herramienta MCP

Campo	Descripción
Actor principal	Agente compatible con MCP
Propósito	Permitir análisis desde asistentes de programación
Precondición	Servidor MCP iniciado por stdio y perfil disponible
Flujo principal	El agente llama <code>analyze_query(query, profile)</code> ; el servidor usa el perfil local y devuelve reporte
Alternativas	Si no se indica perfil, se usa el perfil por defecto configurado
Resultado	Análisis estructurado consumible por el agente

CU-08. Analizar desde Visual Studio Code

Campo	Descripción
Actor principal	Usuario VS Code
Propósito	Analizar la consulta seleccionada desde el editor
Precondición	Extensión instalada y backend local disponible
Flujo principal	El usuario selecciona SQL; ejecuta comando; la extensión invoca backend/API local; muestra resultado
Alternativas	Si el backend no está disponible, la extensión intenta iniciarlo o informa el problema
Resultado	Reporte visible desde el flujo de trabajo del editor

3. Modelo Lógico

a) Análisis de Objetos

Objeto	Responsabilidad	Atributos relevantes
ConnectionConfig	Representar configuración de conexión validada	engine , host , port , database , username , password , extra
ProfileConfig	Persistir datos de perfil local	nombre, motor, conexión, valores por defecto
ConfigManager	Administrar perfiles y cifrado	perfiles, perfil activo, rutas de configuración
AdapterRegistry	Registrar y crear adaptadores	catálogo de motores y clases
BaseAdapter	Definir contrato común de motores	connect , disconnect , execute_explain , get_metrics
Adaptador concreto	Ejecutar lógica específica del motor	driver, parser, métricas y plan original
PlanNode	Representar nodo del plan normalizado	node_type , cost , estimated_rows , actual_rows , actual_time_ms , children , properties
AIAnalysisResult	Contener interpretación opcional de IA	summary , observations , recommendations , suggested_query , raw_response
QueryAnalysisReport	Agrupar evidencia factual del análisis	engine , query , execution_time_ms , plan_tree , plan_summary , raw_plan , metrics , analyzed_at , ai_analysis
ReportSerializer	Exportar e importar reportes	JSON, Markdown y diccionarios
ConnectionDiagnostics	Ejecutar diagnóstico progresivo	estado, duración, mensaje seguro
HistoryManager	Guardar historial local de análisis	reportes por perfil y límites de retención
API Router	Exponer endpoints REST	rutas bajo /api/v1/analyzer
MCP Server	Exponer herramienta para agentes	analyze_query(query, profile)

b) Diagrama de Actividades con objetos

```
flowchart TD
    A["Usuario: consulta + perfil"] --> B["ConfigManager: obtener perfil"]
    B --> C["ConnectionConfig: validar datos"]
    C --> D["AdapterRegistry: crear adaptador"]
    D --> E["BaseAdapter: connect"]
    E --> F["Motor BD: ejecutar EXPLAIN/equivalente"]
    F --> G["Adaptador concreto: parsear salida"]
    G --> H["PlanNode: construir árbol si aplica"]
    G --> I["metrics/raw_plan: conservar evidencia"]
    H --> J["QueryAnalysisReport: ensamblar reporte"]
    I --> J
    J --> K["AIAalyzer configurado"]
    K -- "Sí" --> L["AIAnalysisResult: interpretación separada"]
    K -- "No" --> M["Reporte factual sin IA"]
    L --> N["ReportSerializer/Interfaz: presentar o exportar"]
    M --> N
    N --> O["HistoryManager: guardar si corresponde"]
    O --> P["BaseAdapter: disconnect"]
```

c) Diagrama de Secuencia

CU-01. Administrar perfil de conexión

```
sequenceDiagram
    actor Usuario
    participant Interfaz as CLI/TUI
    participant Config as ConfigManager
    participant Crypto as Servicio de cifrado

    Usuario->>Interfaz: Solicita crear/editar/eliminar perfil
    Interfaz->>Usuario: Solicita motor y datos de conexión
    Usuario->>Interfaz: Ingresa engine, host, puerto, base y credenciales
    Interfaz->>Config: Validar ConnectionConfig
    alt Configuración válida
        Config->>Crypto: Cifrar credenciales sensibles
        Crypto-->>Config: Credenciales cifradas
        Config->>Config: Guardar perfil local
        Config->>Interfaz: Perfil actualizado
        Interfaz-->>Usuario: Confirmación segura
    else Configuración inválida
        Config-->>Interfaz: Error de validación
        Interfaz-->>Usuario: Mensaje claro sin secretos
    end
    end
```

CU-02. Diagnosticar conexión

```
sequenceDiagram
    actor Usuario
    participant Interfaz as CLI/TUI/API
    participant Config as ConfigManager
    participant Diagnostico as ConnectionDiagnostics
    participant Registro as AdapterRegistry
    participant Adaptador as BaseAdapter
    participant Motor as Motor de BD
```

```

Usuario->>Interfaz: Solicita diagnosticar perfil/conexión
Interfaz->>Config: Obtener ConnectionConfig
Config-->>Interfaz: Configuración validada
Interfaz->>Diagnostico: Ejecutar diagnóstico progresivo
Diagnostico->>Registro: Verificar motor registrado
Registro-->>Diagnostico: Motor soportado
Diagnostico->>Adaptador: Probar conexión
Adaptador->>Motor: DNS/TCP/autenticación/operación
Motor-->>Adaptador: Resultado de comprobación
Adaptador-->>Diagnostico: Estado técnico
Diagnostico-->>Interfaz: Estado, duración y mensaje sanitizado
Interfaz-->>Usuario: Muestra diagnóstico

```

CU-03. Analizar consulta

```

sequenceDiagram
    actor Usuario
    participant Interfaz as CLI/TUI
    participant Config as ConfigManager
    participant Registro as AdapterRegistry
    participant Adaptador as BaseAdapter
    participant Motor as Motor de BD
    participant Reporte as QueryAnalysisReport
    participant Historial as HistoryManager

    Usuario->>Interfaz: Solicita analizar consulta
    Interfaz->>Config: Obtener perfil seleccionado
    Config-->>Interfaz: ConnectionConfig
    Interfaz->>Registro: create(engine, config)
    Registro-->>Interfaz: Adaptador concreto
    Interfaz->>Adaptador: connect()
    Adaptador->>Motor: EXPLAIN / PROFILE / equivalente
    Motor-->>Adaptador: Plan, métricas o datos disponibles
    Adaptador->>Reporte: Construir reporte factual
    Reporte-->>Interfaz: QueryAnalysisReport
    Interfaz->>Historial: Guardar reporte si aplica
    Interfaz->>Adaptador: disconnect()
    Interfaz-->>Usuario: Mostrar plan, métricas y resumen

```

CU-04. Exportar reporte

```

sequenceDiagram
    actor Usuario
    participant Interfaz as CLI/TUI
    participant Reporte as QueryAnalysisReport
    participant Serializador as ReportSerializer
    participant Archivo as Sistema de archivos

    Usuario->>Interfaz: Solicita exportar reporte
    Interfaz->>Reporte: Obtener reporte actual
    Reporte-->>Interfaz: Datos factuales disponibles
    Interfaz->>Serializador: Serializar a JSON o Markdown
    Serializador-->>Interfaz: Contenido exportable
    Interfaz->>Archivo: Escribir archivo destino
    alt Escritura correcta
        Archivo-->>Interfaz: Archivo guardado
        Interfaz-->>Usuario: Confirma ruta de exportación
    else Error de escritura
        Archivo-->>Interfaz: Error controlado
    end

```

```
Interfaz-->>Usuario: Informa fallo sin perder reporte
end
```

CU-05. Solicitar interpretación IA opcional

```
sequenceDiagram
    actor Usuario
    participant Interfaz as CLI/TUI/API
    participant Reporte as QueryAnalysisReport
    participant IA as AIAalyzer
    participant ResultadoIA as AIAalysisResult

    Usuario->>Interfaz: Activa análisis IA opcional
    Interfaz->>Reporte: Obtener plan, consulta y motor
    Reporte-->>Interfaz: Evidencia factual
    Interfaz->>IA: analyze(plan_json, query, engine)
    alt Proveedor configurado y responde
        IA-->>ResultadoIA: Resumen, observaciones y recomendaciones
        ResultadoIA-->>Interfaz: Interpretación separada
        Interfaz-->>Usuario: Muestra IA junto al reporte factual
    else IA no configurada o falla
        IA-->>Interfaz: Sin resultado IA
        Interfaz-->>Usuario: Mantiene reporte factual sin bloquear
    end
end
```

CU-06. Usar API REST

```
sequenceDiagram
    actor Cliente as Cliente HTTP
    participant API as API REST
    participant Registro as AdapterRegistry
    participant Adaptador as BaseAdapter
    participant Motor as Motor de BD
    participant Reporte as QueryAnalysisReport

    Cliente->>API: POST /api/v1/analyzer/explain
    API->>API: Validar esquema y ConnectionConfig
    API->>Registro: create(engine, connection)
    Registro-->>API: Adaptador concreto
    API->>Adaptador: connect()
    Adaptador->>Motor: EXPLAIN / equivalente
    Motor-->>Adaptador: Plan y métricas
    Adaptador->>Reporte: Construir reporte factual
    Reporte-->>API: Respuesta estructurada
    API->>Adaptador: disconnect()
    API-->>Cliente: JSON sin persistir credenciales
```

CU-07. Usar herramienta MCP

```
sequenceDiagram
    actor Agente as Agente MCP
    participant MCP as MCP Server
    participant Config as ConfigManager
    participant Registro as AdapterRegistry
    participant Adaptador as BaseAdapter
    participant Motor as Motor de BD
    participant Reporte as QueryAnalysisReport
```



```

Agente->>MCP: analyze_query(query, profile)
MCP->>Config: Resolver perfil indicado o perfil por defecto
Config-->>MCP: ConnectionConfig
MCP->>Registro: create(engine, config)
Registro-->>MCP: Adaptador concreto
MCP->>Adaptador: connect()
Adaptador->>Motor: EXPLAIN / equivalente
Motor-->>Adaptador: Plan y métricas disponibles
Adaptador->>Reporte: Construir reporte factual
Reporte-->>MCP: Resultado estructurado
MCP->>Adaptador: disconnect()
MCP-->>Agente: Reporte consumible por herramienta

```

CU-08. Analizar desde Visual Studio Code

```

sequenceDiagram
    actor Usuario
    participant VSCode as VS Code Extension
    participant Backend as Backend local/API
    participant API as API REST
    participant Registro as AdapterRegistry
    participant Adaptador as BaseAdapter
    participant Motor as Motor de BD
    participant Reporte as QueryAnalysisReport

    Usuario->>VSCode: Selecciona consulta y ejecuta comando
    VSCode->>Backend: Verificar o iniciar backend local
    Backend-->>VSCode: Backend disponible
    VSCode->>API: Enviar consulta y perfil/conexión
    API->>Registro: create(engine, config)
    Registro-->>API: Adaptador concreto
    API->>Adaptador: connect()
    Adaptador->>Motor: EXPLAIN / equivalente
    Motor-->>Adaptador: Plan y métricas
    Adaptador->>Reporte: Construir reporte factual
    Reporte-->>API: Resultado estructurado
    API->>Adaptador: disconnect()
    API-->>VSCode: Reporte para vista del editor
    VSCode-->>Usuario: Muestra análisis en VS Code

```

d) Diagrama de Clases

```

classDiagram
    class ConnectionConfig {
        +str engine
        +str host
        +int port
        +str database
        +str username
        +str password
        +dict extra
    }

    class AdapterRegistry {
        +register(engine)
        +create(engine, config)
        +list_engines()
        +is_registered(engine)
    }

```

```

}

class BaseAdapter {
    <<abstract>>
    +ConnectionConfig config
    +connect()
    +disconnect()
    +test_connection()
    +execute_explain(query) QueryAnalysisReport
    +get_metrics()
    +get_engine_info()
}

```

```

class ConcreteAdapter {
    +connect()
    +execute_explain(query) QueryAnalysisReport
    +disconnect()
}

```

```

class PlanNode {
    +str node_type
    +float cost
    +int estimated_rows
    +int actual_rows
    +float actual_time_ms
    +list children
    +dict properties
}

```

```

class AIAnalysisResult {
    +str summary
    +list observations
    +list recommendations
    +str suggested_query
    +str raw_response
}

```

```

class QueryAnalysisReport {
    +str engine
    +str query
    +float execution_time_ms
    +PlanNode plan_tree
    +str plan_summary
    +dict raw_plan
    +dict metrics
    +datetime analyzed_at
    +AIAnalysisResult ai_analysis
}

```

```

class ReportSerializer {
    +to_json(report)
    +from_json(text)
    +to_markdown(report)
    +to_dict(report)
}

```

```

class AIAnalyzer {
    +is_configured()
    +analyze(plan_json, query, engine)
}

```

```

AdapterRegistry --> BaseAdapter
BaseAdapter --> ConnectionConfig
ConcreteAdapter --|> BaseAdapter
ConcreteAdapter --> QueryAnalysisReport

```

```
QueryAnalysisReport --> PlanNode
QueryAnalysisReport --> AIAnalysisResult
ReportSerializer --> QueryAnalysisReport
AIAnalyzer --> AIAnalysisResult
```

CONCLUSIONES

1. Query Analyzer resuelve una necesidad real de análisis de consultas en entornos con múltiples motores, reduciendo la fragmentación de comandos y formatos.
2. La especificación final conserva el enfoque factual del proyecto: reportar datos del motor, mantener el plan original y evitar puntuaciones universales.
3. La arquitectura por adaptadores permite soportar 13 motores sin acoplar las interfaces de usuario a drivers específicos.
4. Los perfiles locales, cifrado y sanitización de mensajes atienden los principales riesgos de seguridad de credenciales.
5. Las interfaces CLI, TUI, API REST, MCP y VS Code amplían la utilidad del sistema para estudiantes, desarrolladores, administradores y agentes de programación.
6. La IA opcional aporta interpretación, pero no altera el reporte factual ni bloquea el análisis cuando no está configurada.

RECOMENDACIONES

1. Mantener sincronizados FD01, FD02, FD03, FD04 y FD05 cuando se agreguen nuevos motores o cambien contratos de análisis.
2. Documentar por motor el significado exacto de métricas específicas para evitar comparaciones incorrectas.
3. Ampliar escenarios de uso con ejemplos reales antes/después de optimizaciones.
4. Mantener la API REST enlazada a localhost por defecto mientras no exista autenticación formal para exposición en red.
5. Continuar reforzando pruebas de sanitización de secretos y errores públicos.
6. Agregar migraciones explícitas si el esquema de historial local cambia en versiones futuras.

BIBLIOGRAFÍA

1. Bass, L., Clements, P., & Kazman, R. (2021). *Software Architecture in Practice*.
2. Fowler, M. (2002). *Patterns of Enterprise Application Architecture*.
3. Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns*.
4. Kleppmann, M. (2017). *Designing Data-Intensive Applications*.
5. Pressman, R. S., & Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach*.
6. Sommerville, I. (2016). *Software Engineering*.
7. ISO/IEC 25010:2011. *Systems and software quality models*.
8. Ley N.º 29733. Ley de Protección de Datos Personales del Perú.

WEBGRAFÍA

- Python: <https://docs.python.org/3/>
- uv: <https://docs.astral.sh/uv/>
- Pydantic: <https://docs.pydantic.dev/>
- Typer: <https://typer.tiangolo.com/>
- Rich: <https://rich.readthedocs.io/>
- Textual: <https://textual.textualize.io/>

- FastAPI: <https://fastapi.tiangolo.com/>
- PostgreSQL EXPLAIN: <https://www.postgresql.org/docs/current/using-explain.html>
- MySQL EXPLAIN: <https://dev.mysql.com/doc/refman/8.0/en/explain.html>
- MongoDB Explain Results: <https://www.mongodb.com/docs/manual/reference/explain-results/>
- Neo4j Execution Plans: <https://neo4j.com/docs/cypher-manual/current/planning-and-tuning/execution-plans/>
- GitHub Actions: <https://docs.github.com/actions>
- Visual Studio Code Extension API: <https://code.visualstudio.com/api>
- Model Context Protocol: <https://modelcontextprotocol.io/>